STUDENT PLACEMENT MANAGEMENT SYSTEM

Prepared by : Polanati Gayathri Tejaswini

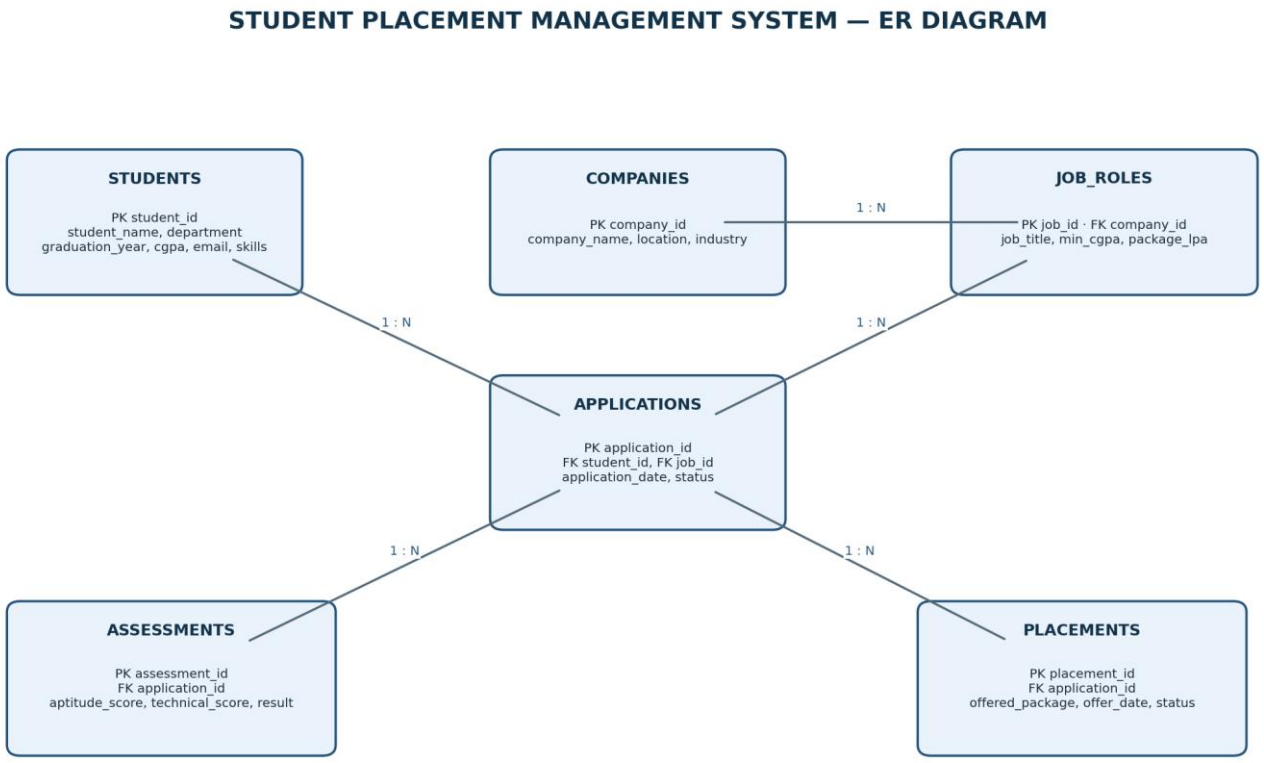
Theme

A college placement cell wants to maintain student details, company information, job roles, applications, assessment results, and placement offers in one structured database. This project uses MySQL to store related information and generate useful reports for placement activities.

Modules

- Students — student profile, department, CGPA and skills
- Companies — company name, location and industry
- Job Roles — job title, eligibility CGPA and package
- Applications — student applications and current status
- Assessments — aptitude and technical scores
- Placements — offer package, date and acceptance status

ER Diagram



PK = Primary Key; FK = Foreign Key. One company can offer many job roles; one student can submit multiple applications; each application can have an assessment and a placement record.

Sample Data

STUDENTS

ID	Name	Dept.	Year	CGPA	Email	Skills
1	Ananya Rao	CSE	2026	8.60	ananya.rao@example.com	Python, SQL
2	Rahul Kumar	ECE	2026	7.80	rahul.kumar@example.com	SQL, Excel
3	Meghana Reddy	CSE	2026	9.10	meghana.reddy@example.com	Java, SQL
4	Arjun Varma	IT	2026	7.20	arjun.varma@example.com	HTML, SQL
5	Sravya Patel	CSE	2026	8.00	sravya.patel@example.com	Python, Java
6	Kiran Das	EEE	2026	6.90	kiran.das@example.com	Excel, Communication
7	Nisha Sharma	IT	2026	8.40	nisha.sharma@example.com	Python, Power BI
8	Vivek Raju	ECE	2026	7.50	vivek.raju@example.com	C, SQL

COMPANIES

Company ID	Company Name	Location	Industry
1	TechNova	Hyderabad	Information Technology
2	DataBridge	Bengaluru	Analytics
3	CloudPeak	Chennai	Cloud Services
4	NextGen Systems	Hyderabad	Information Technology

JOB_ROLES

Job ID	Company ID	Job Title	Min CGPA	Package (LPA)
101	1	Junior SQL Developer	7.00	4.50
102	2	Data Analyst Trainee	7.50	5.00
103	3	Support Engineer	6.50	3.80
104	1	Python Developer Trainee	8.00	5.50
105	4	Graduate Engineer Trainee	7.00	4.20
106	2	Business Intelligence Intern	8.00	4.80

APPLICATIONS

App ID	Student ID	Job ID	Date	Status
1001	1	101	2026-01-10	Selected
1002	2	101	2026-01-11	Interview
1003	3	102	2026-01-12	Selected
1004	4	103	2026-01-13	Applied
1005	5	104	2026-01-14	Rejected
1006	6	103	2026-01-15	Applied
1007	1	104	2026-01-16	Interview
1008	7	106	2026-01-17	Assessment
1009	8	105	2026-01-18	Applied
1010	3	104	2026-01-19	Selected
1011	5	102	2026-01-20	Interview

ASSESSMENTS

Assessment ID	Application ID	Aptitude	Technical	Result
1	1001	85	88	Passed
2	1002	72	78	Pending
3	1003	92	94	Passed
4	1005	65	70	Failed
5	1008	88	82	Passed
6	1010	90	91	Passed

PLACEMENTS

Placement ID	Application ID	Package (LPA)	Offer Date	Status
1	1001	4.50	2026-02-01	Accepted
2	1003	5.00	2026-02-05	Offered
3	1010	5.50	2026-02-08	Offered

Queries

1. Find students with CGPA above 8

```
SELECT * FROM students WHERE cgpa > 8.0;
```

2. Display CSE students

```
SELECT * FROM students WHERE department = 'CSE';
```

3. Show companies located in Hyderabad

```
SELECT * FROM companies WHERE location = 'Hyderabad';
```

4. Display selected applications

```
SELECT * FROM applications WHERE application_status = 'Selected';
```

5. Find job roles with package above 4 LPA

```
SELECT * FROM job_roles WHERE package_lpa > 4.0;
```

6. Count total students

```
SELECT COUNT(*) AS total_students FROM students;
```

7. Count students department-wise

```
SELECT department, COUNT(*) AS student_count FROM students GROUP BY department;
```

8. Calculate average CGPA

```
SELECT ROUND(AVG(cgpa),2) AS average_cgpa FROM students;
```

9. Find highest job package

```
SELECT MAX(package_lpa) AS highest_package_lpa FROM job_roles;
```

10. Count applications by status

```
SELECT application_status, COUNT(*) AS total FROM applications GROUP BY application_status;
```

11. Display student names and job roles applied for

```
SELECT s.student_name, j.job_title FROM applications a JOIN students s ON a.student_id=s.student_id JOIN job_roles j ON
a.job_id=j.job_id;
```

12. Display job titles with company names

```
SELECT j.job_title, c.company_name FROM job_roles j JOIN companies c ON j.company_id=c.company_id;
```

13. Display students who received placement offers

```
SELECT s.student_name, p.offered_package_lpa FROM placements p JOIN applications a ON p.application_id=a.application_id
JOIN students s ON a.student_id=s.student_id;
```

14. Count job roles for each company

```
SELECT c.company_name, COUNT(j.job_id) AS job_count FROM companies c LEFT JOIN job_roles j ON c.company_id=j.company_id
GROUP BY c.company_id,c.company_name;
```

15. Find students who have not applied

```
SELECT s.student_name FROM students s LEFT JOIN applications a ON s.student_id=a.student_id WHERE a.application_id IS NULL;
```

16. Students with CGPA above average

```
SELECT student_name, cgpa FROM students WHERE cgpa > (SELECT AVG(cgpa) FROM students);
```

17. Count placement offers

```
SELECT COUNT(*) AS total_placements FROM placements WHERE placement_status IN ('Offered','Accepted');
```

18. Average package company-wise

```
SELECT c.company_name, ROUND(AVG(j.package_lpa),2) AS average_package_lpa FROM companies c JOIN job_roles j ON
c.company_id=j.company_id GROUP BY c.company_id,c.company_name;
```

19. Technical assessment scores above 80

```
SELECT s.student_name, ass.technical_score FROM assessments ass JOIN applications a ON ass.application_id=a.application_id
JOIN students s ON a.student_id=s.student_id WHERE ass.technical_score > 80;
```

20. Count applications per student

```
SELECT s.student_name, COUNT(a.application_id) AS application_count FROM students s LEFT JOIN applications a ON
s.student_id=a.student_id GROUP BY s.student_id,s.student_name;
```

21. Students with more than one application

```
SELECT s.student_name, COUNT(*) AS application_count FROM students s JOIN applications a ON s.student_id=a.student_id GROUP
BY s.student_id,s.student_name HAVING COUNT(*) > 1;
```

22. Find highest offered package

```
SELECT s.student_name, p.offered_package_lpa FROM placements p JOIN applications a ON p.application_id=a.application_id
JOIN students s ON a.student_id=s.student_id ORDER BY p.offered_package_lpa DESC LIMIT 1;
```

23. Placement count by company

```
SELECT c.company_name, COUNT(p.placement_id) AS placement_count FROM companies c JOIN job_roles j ON
c.company_id=j.company_id JOIN applications a ON j.job_id=a.job_id JOIN placements p ON a.application_id=p.application_id
GROUP BY c.company_id,c.company_name;
```

24. Show applications with company and role

```
SELECT s.student_name,c.company_name,j.job_title,a.application_status FROM applications a JOIN students s ON
a.student_id=s.student_id JOIN job_roles j ON a.job_id=j.job_id JOIN companies c ON j.company_id=c.company_id;
```

25. Placement summary by status

```
SELECT placement_status, COUNT(*) AS total FROM placements GROUP BY placement_status;
```

Conclusion

The Student Placement Management System demonstrates how MySQL can organize placement-related information using relational tables and keys. The queries retrieve, filter, join, group and summarize records to support placement-cell reporting.